

FxmeNet User Manual

Plugins processed on a server on your network

Version 0.1.0

Draft of November 10, 2026

FX-Mechanics
https://fx-mechanics.com

Contents

1 Introduction **1**
- 1.1 What FxmeNet is . . . 1
- 1.2 How it works . . . 1
- 1.3 What you see: native editors . . . 2
- 1.4 What FxmeNet is not . . . 2
- 1.5 What you need . . . 2
- 1.6 Terms used in this manual . . . 2

2 Quick start **3**
- 2.1 Get the software . . . 3
- 2.2 Install the plugins on the server . . . 3
- 2.3 Start the server . . . 3
- 2.4 Install the plugin on the client . . . 4
- 2.5 Your first chain . . . 4
- 2.6 Where to go next . . . 4

3 The server **5**
- 3.1 Command line . . . 5
- 3.2 Which plugins the server offers . . . 5
- 3.3 Plugin folders . . . 5
- 3.4 Ports and discovery . . . 6
- 3.5 Sessions . . . 6
- 3.6 Reading the log . . . 6
- 3.7 Running the server as a service . . . 7
- 3.8 Letting a remote `fxmenet-top` change the folders . . . 7

4 Monitoring with `fxmenet-top` **8**
- 4.1 Starting it . . . 8
- 4.2 The header . . . 8
- 4.3 Sessions . . . 9
- 4.4 Plugins and folders . . . 9
- 4.5 Log . . . 9
- 4.6 Keys . . . 9

5 The FxmeNet plugin **10**
- 5.1 The window . . . 10
- 5.2 Connecting . . . 10
- 5.3 Choosing plugins . . . 11
- 5.4 The chain . . . 11
- 5.5 The Parameters tab . . . 11
- 5.6 Automation . . . 11
- 5.7 Settings . . . 12
- 5.8 Statistics . . . 12
- 5.9 Bypass, projects and offline rendering . . . 13
- 5.10 The same project on another computer . . . 13

6 Routing **14**
6.1 The view . . . 14
6.2 Making and removing connections . . . 14
6.3 The default routing . . . 15
6.4 Editing the chain after wiring . . . 15
6.5 Sidechains and auxiliary buses . . . 15
6.6 Latency and parallel paths . . . 16

7 Latency, reliability and performance **17**
7.1 Block size N and network budget D . . . 17
7.2 Choosing the settings . . . 17
7.3 Measured performance . . . 18
7.4 Making the server reliable (Linux) . . . 18
7.5 Making the client reliable . . . 19

8 Tiers: editors and meters **20**
8.1 How the plugin decides . . . 20
8.2 What the local copy does . . . 20
8.3 Keeping the plugin list up to date . . . 20

9 For plugin developers **21**
9.1 The idea . . . 21
9.2 What to do . . . 21
9.3 Checking . . . 22
9.4 The FxmeNet compatible mark . . . 22

10 Troubleshooting **23**
10.1 The plugin does not find the server . . . 23
10.2 The plugin I want is not in the list . . . 23
10.3 A plugin shows tier `T0` when I expected `T1` . . . 23
10.4 Concealed blocks, dropouts or clicks . . . 23
10.5 Silence at start, stop or jump . . . 24
10.6 The audio is late compared with other tracks . . . 24
10.7 Everything stops when the server restarts . . . 24

11 Reference **25**
11.1 Files . . . 25
11.2 Ports . . . 25
11.3 Limits . . . 25
11.4 More documentation . . . 25

12 Network requirements **26**
12.1 Ports . . . 26
12.2 Bandwidth . . . 26
12.3 Security . . . 26

13 Glossary **27**

1 Introduction

1.1 What FxmeNet is

FxmeNet lets your digital audio workstation (DAW) run its effects on another computer. In the DAW you insert one plugin, called FxmeNet. Behind it, a chain of plugins runs on a *server* (the program `fxmenet-server`) on the same local network. The audio goes to the server and comes back; the DAW sees one ordinary plugin, and compensates the delay it reports, as it does for any plugin with latency.

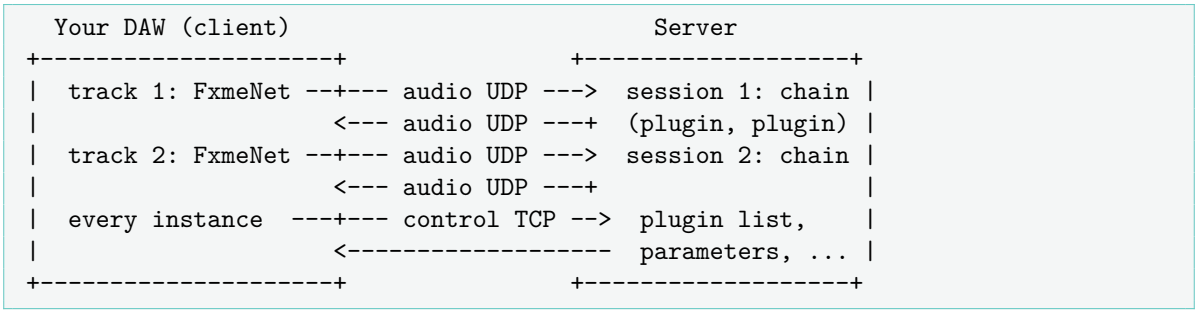
The reason is simple: a session with many heavy plugins can exhaust the CPU of the computer that runs the DAW. A second machine, even an old one, can take that load. In our tests (chapter 7), 24 tracks with 17 plugins each used 75 % of the client's processor when processed locally, and 20 to 25 % through FxmeNet.

The server is a console program. It needs no screen, no desktop session and no keyboard, so it can live in a cupboard or a rack, and it can run on a different operating system than the DAW.

1.2 How it works

FxmeNet has two parts.

- The **client** is the FxmeNet plugin (VST3 and AU), loaded in the DAW. It has the editor you work with.
- The **server** is `fxmenet-server`. It loads the VST3 plugins you ask for and processes the audio.



Each instance of FxmeNet in your project opens one *session* on the server, with its own chain of plugins. Two kinds of traffic flow:

- **Audio**, in small blocks of N samples, over UDP, in both directions. A reply that is late is replaced by a short fade to silence and back (the block is *concealed*); this is how a network that is sometimes slow stays inaudible most of the time.
- **Control**, over TCP: the list of plugins, adding and removing plugins, parameters, saving and restoring state.

The *network budget* D says how many blocks of time a reply has to come back. The latency that FxmeNet adds is about N times D samples, plus what the DAW's own block size adds. Chapter 7 explains how to choose them.

1.3 What you see: native editors

When you work on a plugin that sits on the server, you do not look at a picture of a remote window. If the same plugin is installed on the client computer, FxmeNet opens the plugin's own editor locally, and keeps it in step with the copy on the server. Knobs respond at once, and the editor scales with your screen. If the plugin is made for FxmeNet (see chapter 9), its meters and analysers show the server's real signal. Plugins that are not installed on the client still work, with a generic panel of sliders. These cases are the *tiers* of chapter 8.

1.4 What FxmeNet is not

- It is not a way to play live through a very low latency. The added latency is a few milliseconds at best (a few tens of milliseconds for heavy loads). It suits mixing and production; for live monitoring, keep latency-critical plugins local.
- It is for a local network you control. The traffic is not encrypted and the server does not ask for a password. Do not expose its ports to the internet.
- The server hosts VST3 *effects*. Instruments are not offered.
- It does not make plugins sound different: the server runs the same plugin code your DAW would run.

1.5 What you need

A server. A computer with spare CPU and a wired network connection. Linux is the reference platform (our tests use Ubuntu 24.04). **[TODO: state which other server operating systems are supported]**

A client. A DAW on Windows, macOS or Linux that hosts VST3 (or AU on macOS). The tests used Reaper on Windows. **[TODO: list the DAWs that were tried]**

A network. A wired gigabit network between the two machines. Wi-Fi works badly for audio. A stereo track at 44.1 kHz uses about 3 Mbit/s in each direction (chapter 12).

The plugins. The plugins you want to run must be installed on the server. For their own editors on the client, install the same version there too.

1.6 Terms used in this manual

Block size N Samples sent to the server at a time (32 to 1024).

Network budget D Blocks of time allowed for a reply to come back (1 to 8).

Session One instance of the FxmeNet plugin, connected to the server, with its chain.

Chain The plugins of a session, processed in an order you choose, and optionally wired by the routing (chapter 6).

Concealed block A block whose reply came too late, replaced by a short fade out and in.

A fuller list is in the glossary (chapter 13).

2 Quick start

This chapter takes you from nothing to audio going through a plugin on the server, in about ten minutes.

2.1 Get the software

FxmeNet is built from source for now. You need a C++17 compiler, CMake 3.22 or newer, and the JUCE framework. From the FxmeNet folder:

```
cmake -S . -B build
cmake --build build --target FxmeNetServer -j2
cmake --build build --target FxmeNetPlugin_VST3 -j2
cmake --build build --target FxmeNetTop -j2
```

This gives three programs: the server (`fxmenet-server`), the plugin (`FxmeNet.vst3`) and the monitoring tool (`fxmenet-top`, chapter 4). **[TODO: where the build leaves the files; how JUCE and FxmeTools are found (sibling clones or pinned versions)]**

2.2 Install the plugins on the server

The server offers the VST3 effects it finds in its plugin folders: the standard VST3 folders of the machine (on Linux, typically `~/.vst3`, `/usr/lib/vst3` and `/usr/local/lib/vst3`), plus any you add (section 3.3). Install the plugins you want to use there, the way you would on any computer.

2.3 Start the server

On the server machine, in a terminal:

```
fxmenet-server
```

The first time, the server examines every plugin it finds (this can take a minute for a large collection) and then logs the list, ending with lines like:

```
2026-11-10 14:02:11  48 plugins (48 examined, 0 from the cache, 0 failed)
2026-11-10 14:02:11  listening: control TCP 54322, audio UDP 54321, discovery UDP
    54320
```

Later starts are quick, because only new or changed plugins are examined again. Leave the server running. Stop it with `Ctrl`+`C`.

Note

Your computers' firewall must let the three ports through on the local network (chapter 12). Under Ubuntu's `ufw`, for example, allow 54320 to 54322.

2.4 Install the plugin on the client

Copy `FxmeNet.vst3` to your VST3 folder on the computer that runs the DAW (on Windows, `C:\Program Files\Common Files\VST3`), then let your DAW rescan its plugins. On macOS

the plugin is also built as an Audio Unit. The AU is not notarized, so macOS may refuse to load it at first; the `README` on the project's GitHub page explains what to do. The AU has not been tested yet, and feedback is welcome.

2.5 Your first chain

1. In the DAW, add FxmeNet to a track, as an effect.
2. Click Find. The server answers on the local network; choose it in the list. (If it does not appear, type its address in the field, for example `192.168.1.20`.)
3. Click Connect. The status line next to the button says when the session is ready.
4. The list SERVER PLUGINS shows what the server offers. Type a few letters of a plugin's name in the Filter field, and press `Return` (or double-click a plugin) to add it to the chain.
5. Play. The audio goes through the server. The plugin's controls appear in the right-hand panel; its parameters are also listed in the DAW as automation parameters.

The line at the bottom of the editor shows the latency that FxmeNet reports to the DAW and the count of concealed blocks. On a healthy network the count stays at zero, or nearly.

Tip

To see what the server is doing, open `fxmenet-top` in a terminal on the server (chapter 4). It shows each session, the server's load and the traffic.

2.6 Where to go next

- Choosing the block size and budget, and making the server reliable: chapter 7.
- Parallel chains, sidechains and multichannel plugins: chapter 6.
- Something does not work: chapter 10.

3 The server

`fxmenet-server` is a console program with no window. It hosts plugins for any number of clients at once, as far as the CPU allows.

3.1 Command line

```
fxmenet-server [--name NAME] [--control-port N] [--udp-port N]
               [--discovery-port N] [--no-discovery]
               [--plugin-path DIR]... [--no-default-paths] [--rescan]
               [--remote-admin]
```

Option	Meaning
`--name NAME`	The name shown to clients. Default: the computer's name.
`--control-port N`	TCP port for control. Default 54322.
`--udp-port N`	UDP port for audio. Default 54321.
`--discovery-port N`	UDP port answering Find. Default 54320.
`--no-discovery`	Do not answer Find. Clients must then type the address.
`--plugin-path DIR`	Also search this folder, for this run only. May be repeated.
`--no-default-paths`	Do not search the standard VST3 folders, for this run only.
`--rescan`	Ignore the plugin cache and examine every plugin again.
`--remote-admin`	Let `fxmenet-top` change the plugin folders from other machines (section 3.8).

The server logs to its standard output, with a time stamp on each line, so a service manager (such as systemd's journal) collects it.

3.2 Which plugins the server offers

The server offers VST3 *effects*. Instruments are skipped. A plugin bundle that holds several plugins (a suite) offers each of them.

Each plugin file is examined in a separate process. A plugin that crashes or hangs while it is loaded is logged, skipped, and does not take the server down. It stays skipped until the file changes. The results are kept in a cache file (`fxmenet-server-plugins.xml`, in the `FX-Mechanics` folder of the user's configuration directory), keyed by the file's path, size and date, so only new or changed plugins are examined at the next start. This examination also notes whether the plugin is made for FxmeNet (chapter 9).

If the same plugin is installed in two folders, the first one found is used.

3.3 Plugin folders

The server searches three kinds of folders:

Default folders. The standard VST3 folders of the operating system. They can be turned off.

Saved folders. Folders you add with `fxmenet-top` (chapter 4). The server remembers them from one run to the next, in `fxmenet-server-settings.json` beside the cache file.

This run's folders. Those given with `--plugin-path`. They are not remembered.

Folders are searched with their subfolders. After a change of folders, or when you install a plugin while the server runs, ask for a rescan in `fxmenet-top`. The scan runs in the background; the sessions in progress keep playing. A client sees the new list the next time it connects.

3.4 Ports and discovery

The server listens on three ports (chapter 12). Find in the plugin broadcasts a question on the discovery port, and every server answers with its name, system and number of sessions. Broadcasts normally stay inside one network segment: across routers or VPNs, type the address. Only one server can run on a machine with discovery on; a second one says so in its log and runs without it.

3.5 Sessions

A session starts when a client connects and ends when it says goodbye. If a client's connection drops (a network hiccup, a DAW that crashes), the server keeps the session for 10 seconds, so that the client can resume it without losing its chain.

Each session has its own audio thread. The server asks the operating system to run it with real-time priority; if the system refuses, the server logs a warning and runs it as an ordinary thread, which is less reliable under load (section 7.4).

3.6 Reading the log

At startup the log lists the plugins found. Every 10 seconds, for each session that is working, it prints one line of figures (the numbers below are an example):

```
session 3: 8123 blocks, lost 0, stale 0, from cache 0, queue full 0,
send errors 0, resets 1 | process 412 us, max 10 s 1180 us, max 2310 us |
turnaround max 10 s 1650 us, over 1 block 0, over 2 blocks 0 |
chain latency 256, telemetry frames 240, parameter changes 12
```

Counts are since the session started; the values marked *10 s* cover the last interval.

Figure	Meaning
blocks	Blocks processed.
lost	Blocks that never arrived in time: the server skipped them (the client conceals them).
stale	Late blocks too old to answer, dropped.
from cache	Duplicate requests answered from the server's cache of recent replies.
queue full	Requests refused because the session could not keep up.
send errors	Replies the system refused to send, typically because the send buffer was full (section 7.4).
resets	Times the client asked to restart the chain's audio (transport start, stop or jump).
process	Time spent processing the last block, then the worst in the last 10 seconds, then the worst since the start.

Figure	Meaning
turnaround	Time from a request's arrival to its reply leaving, worst of the last 10 seconds.
over 1 block, over 2 blocks	Replies that left more than one (two) block durations after the request arrived: the server fell behind.
chain latency	Latency of the chain, in samples.

A *process* time that gets close to the block's duration (2.9 ms for 128 samples at 44.1 kHz) means the server is running out of CPU for that session.

3.7 Running the server as a service

Below is an example of a `systemd` unit for Linux. Running the server as a service is a possibility that has not been tested yet, so treat the example as a starting point. Feedback is welcome, and so are equivalents for other systems (launchd on macOS, a Windows service).

```
[Unit]
Description=FxmeNet server
After=network-online.target
Wants=network-online.target

[Service]
ExecStart=/usr/local/bin/fxmenet-server
User=fxmenet
Group=audio
Restart=on-failure
LimitRTPRIO=95
LimitMEMLOCK=infinity

[Install]
WantedBy=multi-user.target
```

3.8 Letting a remote `fxmenet-top` change the folders

`fxmenet-top` can watch a server from any machine of the network. It may change the plugin folders only when it connects from the server's own machine, unless you start the server with `--remote-admin`. Because the network is not authenticated, use this option only on a network you trust.

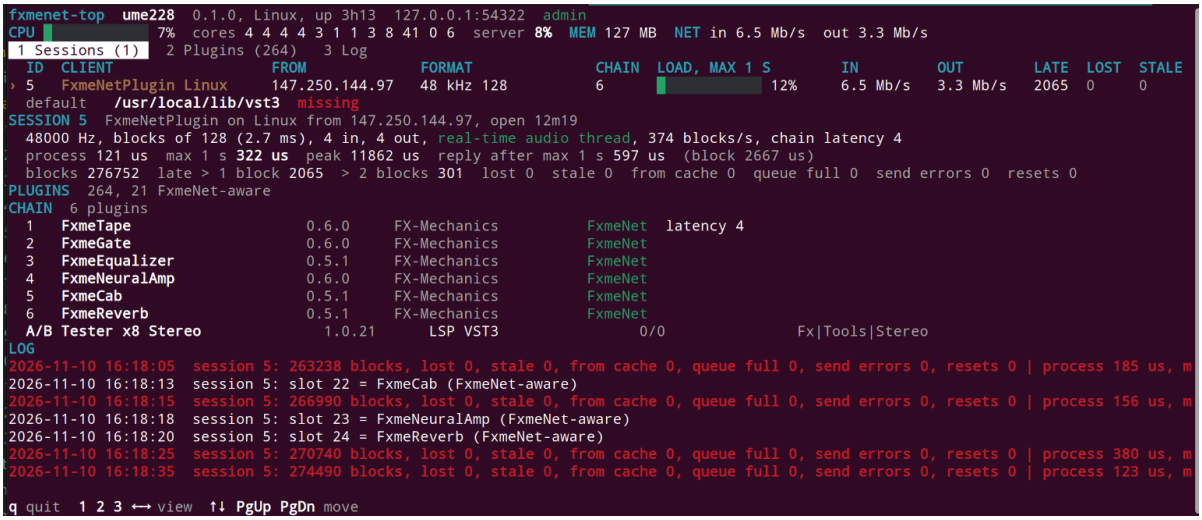
4 Monitoring with `fxmenet-top`

`fxmenet-top` is a terminal program that shows what a server is doing, in the manner of `top`. It runs on Linux, macOS and Windows 10 or later, in any terminal that understands ANSI escape sequences. It connects to the server's control port as a monitor: it opens no session and never touches the audio, so it can be started and stopped at will.

4.1 Starting it

```
fxmenet-top                          # the server on this machine
fxmenet-top 192.168.1.20             # a server elsewhere (port 54322)
fxmenet-top studio.local:54322       # with a port
fxmenet-top --once                   # print one frame without colours, and exit
fxmenet-top --find                   # list the servers on the local network
```

If the server stops, `fxmenet-top` says so and reconnects when it returns. To disable colours, set the environment variable `NO_COLOR`.



4.2 The header

Two lines are always visible. The first gives the server's name, version, system and uptime, the address, and whether you may change the plugin folders (*admin*) or only watch (*read only*). The second gives:

- **CPU**: the machine's total use, each core, and the server process's use (100 % is one core fully used, as in `top`). Windows does not give the per-core figures.
- **MEM**: the server process's memory.
- **NET**: the audio traffic received and sent.

Below them, three views are selected with 1, 2 and 3 (or the left and right arrows).

4.3 Sessions

1. One line per client session: its name and system, its address, the format (sample rate and block size), the number of plugins, a bar for the load, the traffic, and the counts of late, lost and stale blocks. The load bar compares the worst processing time of the last second with the block's duration: green is comfortable, yellow is getting tight, red means the server is close to missing deadlines. A count that has just moved turns red for ten seconds.

Choose a session with the up and down arrows. Under the list, the details show the format, whether the audio thread has real-time priority, the processing times, all the counters, and the session's chain with each plugin's version, vendor and latency. The latest log lines fill the rest of the screen.

4.4 Plugins and folders

2. The folders the server searches (default, saved, and given on the command line), with missing ones marked; the progress of the scan in progress; and the plugins found, sorted by name. Press / to filter them, as in the plugin's own list; Esc clears the filter.

When you are allowed to (*admin*), these keys change the folders:

Key	Action
a	Add a folder (type an absolute path on the server, then Return).
d	Remove the selected saved folder.
x	Turn the default VST3 folders on or off.
r	Rescan: new or changed plugins only.
R	Full rescan, ignoring the cache.
Tab	Move between the list of folders and the list of plugins.

The scan runs in the background and its progress appears at once. Folders you add are kept by the server for its next runs.

4.5 Log

3. The server's log, with the latest lines at the bottom. Scroll with the arrows, PgUp and PgDn; End follows the newest line again. Lines that contain *error*, *warning* or *skipped* are red. / filters the lines.

4.6 Keys

Key	Action
1 2 3 or arrows	Choose the view.
Up Down PgUp PgDn	Move in the list.
Home End	First or last entry (in the log, the newest).
/	Filter the plugins or the log.
Esc	Clear the filter, or cancel typing.
q	Quit.

5 The FxmeNet plugin

This chapter walks through the plugin's window, then explains how it works with the DAW: automation, saving projects, bypass and latency.

5.1 The window



From top to bottom:

Top bar. The FX-Mechanics logo and version, a ? switch that turns the hover help on or off for all instances, and an information button.

Server row. The server's address, Find and Connect (the button reads Disconnect while connected), and the connection status.

Left column. SERVER PLUGINS, the list of what the server offers, with its filter and the Add button; and CHAIN, your plugins in processing order, with Up, Down, Bypass and Remove.

Right side. Four tabs: Parameters, Mapping, Routing and Settings.

Bottom line. Statistics (section 5.8).

5.2 Connecting

Type the server's address (a name or an IP address, with `:port` if it does not use the default) or press Find and choose a server from the list that appears. Connect opens the session. While the plugin is not connected, the audio passes through untouched. The plugin remembers the last server you used, on this computer, and connects again when you reopen a project.

If the connection drops, the plugin reconnects by itself and resumes its session when the server still has it (section 5.10).

5.3 Choosing plugins

SERVER PLUGINS lists what the server offers. Each row shows the plugin's name, its vendor, its input and output channels, and a tag T2, T1 or T0 that says how the plugin can be shown on this computer (chapter 8). Hover over a row to see why it has its tier.

The list is sorted by tier (T2 first, then T1, then T0) and by name inside each tier. While the plugin scans this computer for the first time, the order is by name only, and the tags read T?.

To find a plugin quickly, type a few letters in the Filter field above the list. Only plugins whose name contains every word you type remain. The first match is selected: press `Return` to add it. `Esc` clears the filter. Add (or a double-click) adds the selected plugin at the end of the chain.

5.4 The chain

CHAIN lists your plugins in the order the server processes them, from top to bottom. Select one to see its controls on the right. Buttons under the list:

Up, Down Move the selected plugin in the chain.

Bypass Bypass the selected plugin, on the server.

Remove Remove it. The state of the plugin is lost.

A plugin that cannot be loaded on the server shows the reason in red (a missing licence, a plugin that failed to start, ...).

The routing between the plugins is serial by default, as in a plain chain. To do parallel processing or sidechains, see chapter 6.

5.5 The Parameters tab

The tab shows the selected plugin.

- If the plugin is installed on this computer in the same version (tier T1 or T2), you see its own editor. It is a local copy, kept in step with the server: what you change in it is sent to the server, and what changes on the server appears in it. The window grows to fit the editor and returns to its normal size when you leave it.
- Otherwise, you see generic controls: one row per parameter with its name, a slider and its value. Double-click a slider to return to the default.

A button above the controls switches between the plugin's own editor and the generic controls. A last touched button next to it names the control you moved last in the plugin's editor, with a menu to map it to an automation parameter (below).

5.6 Automation

The DAW sees FxmeNet as one plugin with 256 automation parameters, named `p001` to `p256` until they are used. When a plugin joins the chain, its automatable parameters take the first free entries, and the DAW shows them under the plugin's name (for example "FxmeCompressor: Ratio"). You can automate them, and the DAW's own controls, as with any plugin.

The Mapping tab lists every entry in use and what it controls. To free entries, select them and press Unmap, or use Unmap all.

To map a parameter by hand, use one of the two methods:

- In the generic controls, press Map on a row to give it the first free entry.
- Press Learn, then move any of FxmeNet's parameters in the DAW: the parameter you moved becomes the one that controls it.

Note

Automation lanes in the DAW belong to the numbered entries. When you remove a plugin, its entries stay free but keep their lane. FxmeNet does not give a used entry to the next plugin you add (unless you map it by hand or all 256 have been used), so a recorded automation cannot silently drive another plugin's parameter.

5.7 Settings

The Settings tab has three parts.

Stream. Block size N and Network budget D, with the latency they give (chapter 7).

Transport. Silence in-flight audio at reset: when the DAW restarts playback or jumps, drop the audio still on its way back from the server, so you hear silence for the length of the latency (as a local plugin clearing its delay line) rather than the tail of the previous position. Log transport events writes what the audio thread sees around transport events, concealed blocks included, to `fxmenet-client-events.log` in the FX-Mechanics settings folder; it is for diagnosing problems.

Local plugins. The scan of the plugins installed on *this* computer, used for the plugins' own editors. Rescan looks for new or changed plugins; Full rescan examines everything again; Folders... adds folders to search beside the standard VST3 ones.

5.8 Statistics

The line at the bottom of the window gives:

Latency What FxmeNet reports to the DAW, in samples and milliseconds.

Round trip The last measured time for a message to reach the server and come back.

Concealed Blocks that came back too late and were replaced.

Rescued Blocks that would have been concealed but arrived in time after a short wait.

UDP dropped Audio messages the client threw away because they could not be put together: pieces missing, damaged or too large. It should stay at zero on a healthy network.

Resets How many times the DAW reset the plugin (playback start, stop or a jump in the timeline). Each time, the server resets its chain at the same point of the stream.

Host ahead How far the DAW processes audio ahead of real time, in milliseconds. It is 0 when the DAW calls the plugin at the pace of the sound card, and larger when the DAW renders with a read-ahead buffer.

Parameters mapped How many of the 256 entries are in use.

A concealed count that rises steadily means the server or the network cannot keep up with the chosen settings (chapter 7).

5.9 Bypass, projects and offline rendering

- **DAW bypass.** When you bypass FxmeNet in the DAW, the plugin passes the audio dry, delayed by the same latency, so that switching it on and off does not move the track in time.
- **Saving.** The project stores the server, the settings, each plugin with its state, the parameter mapping and the routing. When you open the project, the plugin rebuilds the chain on the server and connects. If the server is not reachable then, the plugin keeps the state, and you can connect later.
- **Offline rendering.** When the DAW renders faster than real time (a bounce or an export), the plugin waits for each reply, so nothing is concealed and the result is exact.

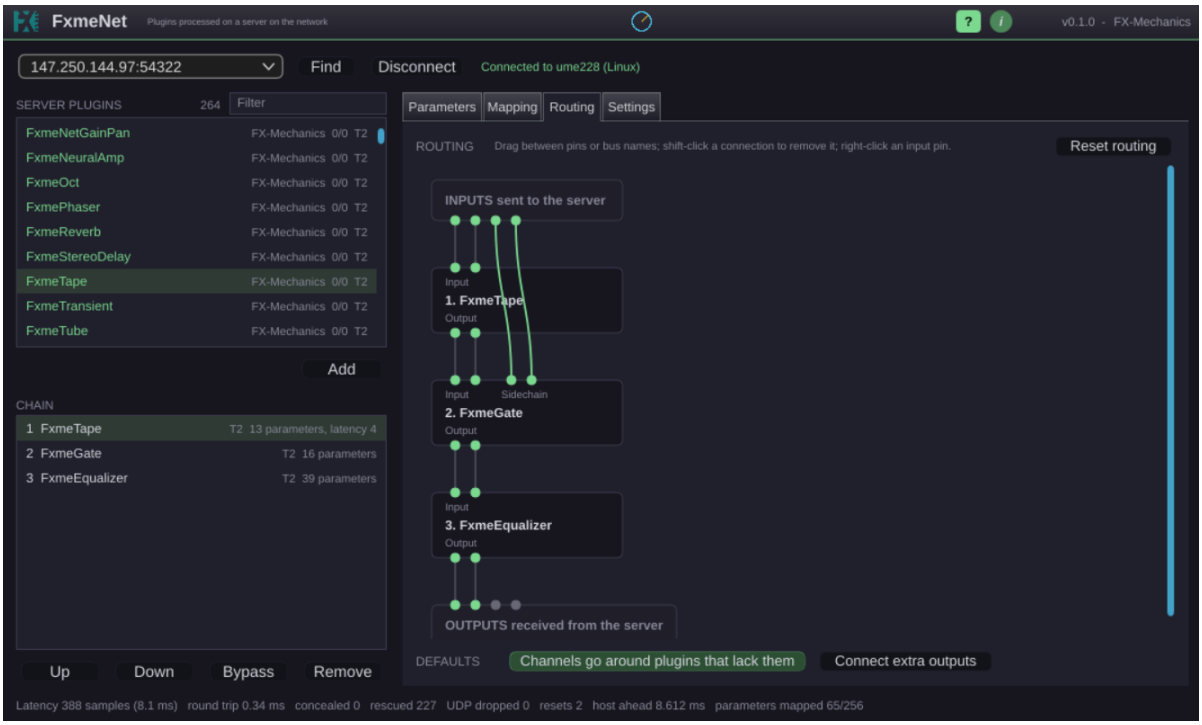
5.10 The same project on another computer

A project that uses FxmeNet needs the server to be reachable and to have the same plugins installed. If a plugin is missing on the server, its row in the chain shows an error and the audio is not processed by it. The audio goes through that position of the chain unchanged, as if the plugin were bypassed, and the rest of the chain keeps working. A future release will let you optionally compute the chain locally on the client when the server is not available.

6 Routing

By default the chain is serial, like the inserts of a mixer channel: the output of each plugin feeds the next one, and the last one feeds the DAW. The Routing tab turns the chain into a patchbay, so that you can run plugins in parallel, feed a sidechain from the DAW or from another plugin, or send the outputs of a multi-output plugin to different channels.

6.1 The view



From top to bottom, the tab shows:

- **Inputs**: the channels sent to the server (the plugin's input channels in the DAW);
- one box for each plugin of the chain, in chain order, with small *pins* for its inputs on top and for its outputs below, grouped by *bus* ("Main", "Sidechain", and so on);
- **Outputs**: the channels received from the server (the plugin's output channels in the DAW).

A connection runs from an output pin to an input pin that is *below* it. Connections cannot go back up, so there is no feedback loop.

6.2 Making and removing connections

- **Connect:** drag from an output pin to an input pin.
- **Connect a whole bus:** drag from a bus name to another bus name (or to a pin). The channels connect in order, so a stereo link takes one drag.
- **Remove:** click a connection while holding `Shift`.

- **Right-click an input pin** to disconnect it, or to put it back to the default rule (Back to the default routing).
- Reset routing puts the default routing back.

An output can feed several inputs (each gets a copy). Several outputs on one input are summed, as in a DAW; this is how parallel paths are mixed back together.

A bypassed plugin keeps its pins and passes each input channel to the output channel with the same number.

6.3 The default routing

Until you change anything, each input of a plugin is fed by the nearest plugin above it that has a matching output channel (the Inputs count as the top box). This reproduces the behaviour of an ordinary serial chain:

- a plugin with fewer channels than the stream lets the other channels pass around it to the next plugin that has them;
- a mono plugin with two outputs gives the plugin below its second output, not the DAW's second channel;
- sidechain and other auxiliary inputs start unconnected.

Two options under DEFAULTS change the default rule. They never affect connections you made by hand.

Channels go around plugins that lack them On by default. Off: each plugin is fed only by the one right above it, so channels that no plugin processes are silent.

Connect extra outputs Off by default. On: the extra outputs of a plugin continue its main ones, so a plugin with two main and two extra outputs feeds four channels below it.

6.4 Editing the chain after wiring

You can add, remove and move plugins after you have wired the chain. Connections you did not touch follow the default rule in the new chain. For the ones you made by hand:

- when a plugin is removed, what came out of it is replaced by what fed the same channel of its input, as if it were bypassed for good;
- when a plugin is moved, connections that no longer run downward are dropped;
- an input that loses all its sources through an edit goes back to the default, so the audio keeps flowing. An input you emptied yourself stays empty.

6.5 Sidechains and auxiliary buses

A sidechain input only becomes active when you connect something to it: connect a DAW channel (an input) to the plugin's sidechain bus to compress a bass with a kick, for example. The server then enables that bus, which prepares that plugin again; expect a brief gap on that plugin only. Unconnected auxiliary buses stay disabled, because many plugins behave differently with an enabled bus that carries silence.

The channels the plugin sends to the server are the ones its connections need. Channels you do not connect are not sent, which saves network bandwidth.

6.6 Latency and parallel paths

When two paths of different latency meet at an input, FxmeNet delays the shorter path so that they stay aligned. A parallel compressor, for example, keeps its dry and wet signals in phase even when the processed path goes through a linear-phase equaliser. The latency the DAW is told is the one of the longest path from the inputs to the outputs.

7 Latency, reliability and performance

FxmeNet trades latency for reliability. This chapter explains the two settings that control the trade, how to read what happens, and how to tune the machines.

7.1 Block size N and network budget D

The plugin sends the audio to the server in blocks of N samples and gives each reply D blocks of time to come back. Both are in the Settings tab.

Block size N 32 to 1024 samples. A small block means less latency but more packets per second and tighter deadlines for the server and the network.

Network budget D 1 to 8 blocks. A larger budget survives a slower network or a busier server, at the price of more latency.

The latency that FxmeNet reports to the DAW is roughly N times D samples plus the DAW's own block size, plus the latency of the chain's plugins. The Settings tab shows the exact figure for your settings, in samples and milliseconds, split between the network and re-blocking part and the chain itself. The DAW compensates for all of it, so the track stays in time; what changes is how late a played instrument is heard, which matters when you record through the plugin.

If a reply is not back in time, that block is *concealed*: a short fade to silence (64 samples) and back, so that there is no click, only a very brief dip. Concealed blocks are counted on the bottom line of the window.

The default is N = 128 and D = 2 (5.8 ms of budget at 44.1 kHz).

7.2 Choosing the settings

Situation	Suggested settings
Mixing, many heavy tracks	1024×2 or 512×2: the most robust. Latency is no concern while mixing.
Recording through the plugin, light chain	128×2, or 64×2 if the server has a lot of spare CPU.
A server near full load	Larger blocks. A busy server sometimes stalls for 20 to 50 ms, which small blocks cannot absorb.
Network with other traffic or Wi-Fi	Raise D, or move to a wired connection.

Tip

Test with the DAW's own settings in mind: the DAW's audio buffer adds to FxmeNet's latency. Using a large buffer in the DAW for mixing and a small one for recording is common practice, and works the same here.

7.3 Measured performance

These figures come from the first stress tests (details in the project's `doc/Tests` folder), on old machines: a 2015 Intel Core i7-6700K for the client (Windows, Reaper, an RME ASIO interface at 44.1 kHz and a buffer of 128) and a 2018 AMD Ryzen 5 PRO 2400G for the server (Ubuntu 24.04, four cores). Every track had one FxmeNet instance, and the "full chain" holds all 17 plugins of the FxmeFX collection (FX-Mechanics' effects).

Tracks	Chain	N×D	Result
24 local	full	(local)	75 % of the client's CPU.
24 remote	full	256×2	Client at 25 %, server at about 66 %, about 10 concealed blocks in 5 minutes on one track.
24 remote	full	1024×2	Client under 20 %, almost no concealed blocks.
28 remote	full	1024×2	Server above 75 %: no concealed blocks, or very rare ones. Very usable.
32 remote	one equaliser	128×2	Network at 95 Mbit/s each way, no problem.

What these tests show:

- The offload works: the client's processor load fell from 75 % to 20 to 25 % for the same session.
- Heavy loads (28 tracks with 17 plugins each, 476 plugin instances) are very usable with large blocks.
- Small blocks need headroom. While the server has spare CPU, 128 and 256 work well; close to full load the server stalls for a few tens of milliseconds now and then, which short deadlines cannot absorb.
- These are extreme loads. Real sessions rarely put 17 plugins on every track.

7.4 Making the server reliable (Linux)

A generic Linux system works. These are the usual steps for audio on Linux; try them one at a time and check after each whether the concealed count goes down. More testing will be done. Feedback is welcome on the performance you get on other systems and with other optimizations.

1. **Real-time priority for the audio threads.** Check with

   ```
   ps -eLo tid,cls,rtprio,pcpu,comm | grep -i fxme
   ```

   while a session is open: the audio threads should be in class `RR` or `FF`. If they are `TS`, the system refused real-time priority (the server's log says so). Create `/etc/security/limits.d/audio.conf`:

   ```
   @audio - rtprio 95
   @audio - memlock unlimited
   ```

   Add the account that runs the server to the `audio` group, and log in again (or restart the service).

2. **Bigger socket buffers.** Under heavy load the kernel can drop replies when the send buffer is full (the server's log counts them as *send errors*; `nstat -az UdpSndbufErrors` shows the system-wide count). Raise the limits, then restart the server:

   ```
   sudo sysctl -w net.core.rmem_default=4194304 net.core.rmem_max=8388608
   sudo sysctl -w net.core.wmem_default=4194304 net.core.wmem_max=8388608
   ```

   To keep them after a reboot, put the four lines (without `sudo sysctl -w`) in `/etc/sysctl.d/90-fxmenet.conf`.

3. **CPU frequency.** Set the governor to performance:

   ```
   sudo cpupower frequency-set -g performance
   ```

4. **Kernel.** A low-latency or fully preemptible kernel (`linux-lowlatency`, or booting with `preempt=full`) helps the worst cases.

[TODO: When the server asks for large socket buffers itself (todo item 21), step 2 becomes: "the log tells you if the system limits are too low".]

7.5 Making the client reliable

- Use a wired gigabit connection, and keep other heavy traffic off it.
- In Reaper, try turning off *anticipative FX processing* for the tracks that hold FxmeNet, and compare the concealed counts.
- On the network adapter, turn interrupt moderation off or to low, and use the high-performance power plan.

8 Tiers: editors and meters

Every plugin in the server's list carries a tag, T0, T1 or T2. It says what you can have on the client for that plugin.

Tier	What you get
T2	The plugin's own editor on the client, with its meters, spectra and other displays showing the server's real signal. The plugin is made for FxmeNet (chapter 9) and installed in the same version on the client.
T1	The plugin's own editor on the client, kept in step with the server's copy. Meters and analysers that read the audio inside the plugin stay empty, since the audio is not on the client.
T0	Generic controls: a slider per parameter. Used when the plugin is not installed on the client, or is another version.

8.1 How the plugin decides

The plugin scans the VST3 folders of the computer that runs the DAW. A server plugin matches a local one when the identifier, the name and the vendor are the same. The version then decides:

- the same version, and the server says the plugin is made for FxmeNet: tier T2;
- the same version otherwise: tier T1;
- another version, or no local copy: tier T0.

When both computers run the same operating system, the plugin also compares the binary files. A difference (another build of the same version) is shown on hover but does not stop the plugin from being used. Hover over a row of the list to see the reason for its tier.

The FxmeNet plugin itself is never listed, and neither are instruments.

8.2 What the local copy does

For tiers T1 and T2, the plugin creates a local copy of the server's plugin when you first open its editor. This *mirror* never processes audio; it only shows the editor. Your changes go to the server, and the server's state comes back to the mirror. For a plugin made for FxmeNet, whatever the plugin changes outside its parameters (a preset, a loaded file) is copied to the server in full.

Note

Install the same plugin versions on both machines. A tier T1 or T2 editor on a different version is a source of surprises, so FxmeNet does not use it; the plugin shows as tier T0.

8.3 Keeping the plugin list up to date

If you install a plugin on the client while the DAW runs, use Rescan in the Settings tab, and the tags update. If your plugins live in a folder that is not standard, add it with Folders. . . .

9 For plugin developers

FxmeNet runs any VST3 effect. A plugin can also be made *FxmeNet-aware*, and then it gets tier T2 (chapter 8): its editor on the client shows the live meters, spectra and values of the copy that runs on the server. This chapter summarises what that takes. The reference is the file `fxmenet-howto.md` of FxmeTools, with the demonstration plugin `Examples/GainPan` in the FxmeNet source.

9.1 The idea

The plugin is loaded twice: a *source* on the server, which processes audio, and a *mirror* on the client, which never processes and shows the editor. What the editor reads that is not a parameter (a meter, a spectrum, a computed value) is sent from the source to the mirror as telemetry and written into the mirror's own objects, so the editor itself does not change. Outside FxmeNet the plugin behaves exactly as before.

9.2 What to do

1. Add an `fxme::net::NetBridge` to the processor, and return it from `getVST3ClientExtensions()`.
2. Register everything the editor reads that is not a parameter, with a stable name each: meters, taps, atomics, computed scalars.
3. In the few getters that compute from the DSP, return the telemetry value when the plugin is a mirror.
4. Skip the work that only feeds the DSP when the plugin is a mirror.
5. Call `notifyStateChanged()` when state that is not a parameter changes (a loaded file, for example).

```
#include <FxmeTools/net/NetBridge.h>

class MyProcessor : public juce::AudioProcessor
{
public:
    juce::VST3ClientExtensions* getVST3ClientExtensions() override { return &net; }
private:
    Compressor compressor;
    fxme::net::NetBridge net;   // after everything it registers
};

MyProcessor::MyProcessor()
{
    net.telemetry().addMeter ("gr", compressor.getGrMeter());
}
```

The editor reads	Register
a level meter	`addMeter (name, meter)`

The editor reads	Register
a spectrum or waveform tap	`addTap (name, tap)`
a `std::atomic<float>`	`addValue (name, atomic)`
a function that computes a value	`addScalar (name, getter)`
a read-and-reset function	`addPeak (name, function)`

9.3 Checking

The FxmeNet source includes a test tool, `FxmeNetLoopback`, which loads your plugin twice in one process, as source and mirror, and pumps the telemetry between them. Both editors appear side by side; the mirror's meters must follow the source's.

```
FxmeNetLoopback path/to/MyPlugin.vst3
```

9.4 The FxmeNet compatible mark

Plugins that support FxmeNet may carry the FxmeNet mark, so that users know the plugin gives a tier T2 experience.





[TODO: The two marks above are drafts (a connector-shaped logo and a small icon). Decide the final design and supply it as SVG and as transparent PNG in several sizes. Write the conditions of use: what a plugin must do to carry it (for example, that it passes the Loopback check and that its editor works as a mirror), where to show it (website, manual, plugin's information panel), and the licence of the artwork.]

10 Troubleshooting

10.1 The plugin does not find the server

- Is the server running? On its machine, `fxmenet-top` should show it. If not, start `fxmenet-server`.
- Does Find show nothing? Discovery uses a broadcast, which does not cross routers or most VPNs, and is blocked by some firewalls. Type the server's address in the field instead.
- Can the client reach the server's ports? Check the firewall on the server (TCP 54322, UDP 54321 and 54320) and, on the client, that the DAW is allowed to use the network.
- Is a second server running on the same machine? Only one can answer Find; give the other one a different port set, or no discovery.

10.2 The plugin I want is not in the list

- It must be a VST3 *effect* installed on the server, in a folder the server searches (section 3.3).
- Look at the server's log (or the 3 view of `fxmenet-top`) for "skipped": a plugin that crashed or hung when loaded is left out until it changes.
- You installed it while the server ran: rescan with r in `fxmenet-top`, then reconnect the plugin.
- The list is filtered: clear the Filter field.

10.3 A plugin shows tier T0 when I expected T1

The plugin must be installed on the client, in a folder the client scans, in the same version. Hover over the row for the reason. Use Rescan or Folders… in the Settings tab (chapter 8).

10.4 Concealed blocks, dropouts or clicks

Look at where the problem is.

1. In `fxmenet-top`, is the server's CPU high, or a session's load bar red? Then the server is overloaded: use larger blocks (1024×2), reduce the load, or use a faster server.
2. Does a session's *lost* count rise? Requests do not reach the server in time: look at the network (wired? other traffic? a switch that drops packets?).
3. Does *send errors* rise in the server's log? The kernel drops replies: raise the socket buffers (section 7.4).
4. Do the threads show `TS` instead of `RR` in the check of section 7.4? Give the audio threads real-time priority.
5. Only the client's concealed count rises while the server's counts are zero? Replies are delayed on their way back, or the server stalls briefly. Raise D by one.

If clicks persist, enable Log transport events in the Settings tab, reproduce the problem, and keep `fxmenet-client-events.log` and the server's log together for the report.

10.5 Silence at start, stop or jump

This is the Silence in-flight audio at reset option of the Settings tab doing its job: for the length of the latency after a restart, nothing is heard from the plugin, as with a local plugin whose delay line is cleared. Turn the option off to hear what was still in flight.

10.6 The audio is late compared with other tracks

The DAW compensates for the latency that FxmeNet reports. If a track is still late, check that the DAW's plugin delay compensation is on and that the DAW applies it to this track.

10.7 Everything stops when the server restarts

The plugin reconnects by itself and rebuilds the chain on the server. If the server does not have the same plugins any more, the affected plugins show an error in the chain. While the server is away, the sound becomes dry at once; when the server is back, the processed sound returns. Restarting the server during playback was tested, without audible glitches.

11 Reference

11.1 Files

File	Content
`FX-Mechanics/fxmenet-server-plugins.xml`	The server's plugin cache.
`FX-Mechanics/fxmenet-server-settings.json`	The folders and the default-folders switch set with `fxmenet-top`.
`FX-Mechanics/FxmeNet.settings`	Client: the last server, the hover-help switch, the extra plugin folders (machine-wide).
`FX-Mechanics/fxmenet-client-plugins.xml`	Client: the cache of the local plugin scan.
`FX-Mechanics/fxmenet-client-events.log`	Client: the transport events log, when enabled.

The `FX-Mechanics` folder is in the user's application-data folder (on Linux, `~/.config`; on Windows, `%APPDATA%`; on macOS, `~/Library/Application Support`). **[TODO: verify these locations on each system]**

11.2 Ports

See chapter 12.

11.3 Limits

- 256 automation parameters per FxmeNet instance.
- 1 to 64 channels in and out of each FxmeNet instance (stereo by default).
- Block sizes from 32 to 1024 samples; a network budget of 1 to 8 blocks.
- Plugins: VST3 effects on the server.

11.4 More documentation

The project's `doc` folder holds the technical documents: `architecture.md` (how the code is built), `protocol.md` (the wire protocol, for anyone writing another client or tool), `routing.md` (the design of the routing) and `Tests/stress-tests.md` (the measurements of chapter 7 in full).

12 Network requirements

12.1 Ports

Port	Protocol	Use
54320	UDP	Discovery: answers the plugin's Find. Optional.
54321	UDP	Audio: requests and replies of all sessions, and telemetry.
54322	TCP	Control: the plugin lists, parameters, state, and `fxmenet-top`.

All three can be changed with the server's options (chapter 3). The client opens connections to the server; the server only needs these three ports open to the client.

12.2 Bandwidth

Audio travels as 32-bit floating-point samples, uncompressed, in both directions: about 2.8 Mbit/s for a stereo track at 44.1 kHz, 3.1 Mbit/s at 48 kHz, and the more for more channels. In the first stress tests, 24 stereo tracks used about 71 Mbit/s each way and 32 tracks used 95 Mbit/s each way. A gigabit connection carries several hundred tracks of this kind in theory; in practice, the server's CPU and the quality of the network limit the number of tracks first.

Only the channels that the routing uses are sent (chapter 6).

12.3 Security

The traffic is not encrypted or authenticated. A token in each session keeps stray packets out, but anyone on the network can talk to the server. Use FxmeNet on a network you trust, and keep the ports closed on the internet.

13 Glossary

Aux bus A plugin input or output beyond its main ones, such as a sidechain.

Block A group of N samples sent to the server and returned.

Budget (D) The number of blocks of time a reply has to come back.

Chain The plugins of a session, in processing order.

Client The FxmeNet plugin in the DAW.

Concealed block A block whose reply was late, replaced by a short fade to silence and back.

Control channel The TCP connection that carries plugin lists, parameters and state.

Discovery The broadcast that lets Find locate servers.

Mirror The local copy of a server plugin that shows its editor and never processes audio.

Monitor A connection that watches a server without a session (`fxmenet-top`).

Pin The place where a connection joins a plugin's input or output in the routing view.

Pool The 256 automation parameters of an FxmeNet instance.

Server `fxmenet-server`.

Session One FxmeNet instance connected to the server.

Telemetry The values (meters, spectra) a plugin made for FxmeNet sends from the server to its editor on the client.

Tier T0, T1 or T2: how a plugin can be shown on the client (chapter 8).